
Python 金融数据分析

第 9 章 模拟与期权定价

主讲人： 王伟冠, 吕文俊
院 系： 经济学院金融系

上海大学

2026 年 9 月 24 日



1. 为什么需要模拟
2. 随机数
3. 模拟随机变量：一步到位
4. 模拟随机过程：走出路径
5. 欧式期权的蒙特卡洛估值
6. 小结与练习



1. 为什么需要模拟
2. 随机数
3. 模拟随机变量：一步到位
4. 模拟随机过程：走出路径
5. 欧式期权的蒙特卡洛估值
6. 小结与练习



- 会用 `numpy.random` 生成**随机数** (均匀、正态等分布), 理解随机种子的作用
- 会**模拟**股票价格的未来走势: 一次模拟到期日 (静态) 与逐日模拟路径 (动态)
- 会用**蒙特卡洛方法**为欧式期权定价, 并与 BSM 解析公式对比
- 理解“模拟路径越多, 估计越准”背后的 $1/\sqrt{T}$ 规律

本章主线: 给一份期权”称重量”

一份行权价 105 元的看涨期权值多少钱? 答案不是查表, 而是**模拟出成千上万种未来**, 看每种未来里期权能赚多少, 再取平均。

- 第 8 章的积分、第 3 章的正态分布，今天派上大用场。
- 欧式期权有漂亮的**解析公式**（Black-Scholes-Merton，简称 BSM），一行就能算出价格。
- 但现实中大量产品**没有**解析公式：

什么时候必须靠模拟？

- 收益依赖整条路径（如障碍期权、亚式期权）
 - 可以提前行权（美式期权）
 - 涉及多个相互关联的风险因子
-
- **蒙特卡洛模拟（MCS）**：用“大量随机实验的平均值”逼近理论期望，是金融业最通用的数值定价技术。
 - 本章先用欧式期权练习这套方法——因为它有解析答案，**可以当场检验我们模拟得对不对**。

- 期权到期收益是个**随机变量**：股价涨了才值钱，跌了就归零。
- 金融学告诉我们（风险中性定价）：

$$C_0 = e^{-rT} E^{\mathbb{Q}}[h(S_T)]$$

- $h(S_T) = \max(S_T - K, 0)$ ：到期收益（看涨期权）
- e^{-rT} ：把未来的钱折现回今天
- $E^{\mathbb{Q}}$ ：风险中性世界里的“平均”

零基础提醒

别被 $\mathbb{Q}$ 吓到：你只需记住——**期权价格 = 折现后的平均到期收益**。“平均”算不出来时，就用随机数**抽样模拟**出来！

1. 为什么需要模拟
- 2. 随机数**
3. 模拟随机变量：一步到位
4. 模拟随机过程：走出路径
5. 欧式期权的蒙特卡洛估值
6. 小结与练习



- 所有模拟工作的基础是**伪随机数**，由 `numpy.random` 提供：

```
1 import numpy as np
2 import numpy.random as npr    # 随机数子库
3 np.random.seed(1000)          # 固定种子：结果可重现
4 npr.rand(5)                   # 5个 [0,1) 均匀分布随机数
```

- `rand()` 只给 $[0,1)$ 区间的数，想要其他区间？线性变换即可：

```
1 a, b = 5, 10
2 npr.rand(5) * (b - a) + a    # 变到 [5, 10) 区间
3 # array([6.0617, 5.2035, 6.986 , 6.1657, 9.2087])
```

为什么先 `seed(1000)`？

伪随机数由种子决定。种子相同，整串随机数完全相同——这是课堂复现、作业批改、结果核查的前提。

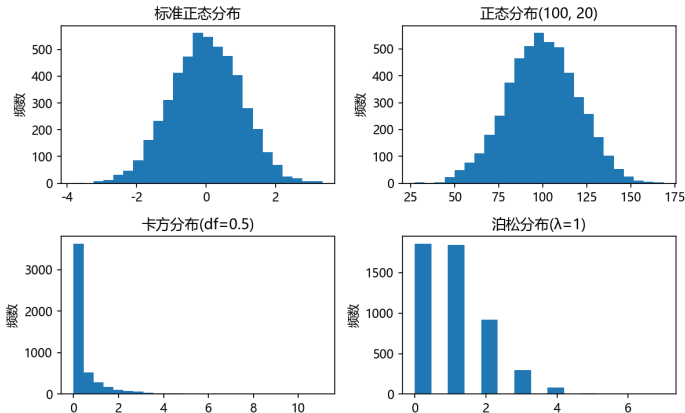
- 金融模型几乎处处用到**标准正态分布**随机数：

```
1 sn = npr.standard_normal(1000000) # 一百万个
2 sn.mean() # -0.001787, 接近理论值 0
3 sn.std() # 1.001300, 接近理论值 1
```

- 接近但不完全等于——这正是后面“方差缩减”要解决的问题。
- 其他常用函数：

随机数生成函数速查

函数	作用
<code>rand(d0, d1, ...)</code>	均匀分布随机数
<code>standard_normal(size)</code>	标准正态分布样本
<code>normal(loc, scale, size)</code>	任意均值/标准差的正态分布
<code>randint(low, high, size)</code>	区间内的随机整数
<code>poisson(lam, size)</code>	泊松分布（模拟罕见事件，如暴跌）



- 左上：标准正态（钟形对称）；右上：正态（100, 20）——形状相同、位置缩放不同。
- 左下：卡方分布（右偏、只取正值，用于利率模型）；右下：泊松分布（离散，模拟**罕见冲击**发生的次数）。

1. 为什么需要模拟
2. 随机数
- 3. 模拟随机变量：一步到位**
4. 模拟随机过程：走出路径
5. 欧式期权的蒙特卡洛估值
6. 小结与练习



- BSM 框架下, 已知今天的股价 S_0 , 到期日 T 的价格满足:

$$S_T = S_0 \exp\left(\left(r - \frac{\sigma^2}{2}\right)T + \sigma\sqrt{T}z\right), \quad z \sim N(0, 1)$$

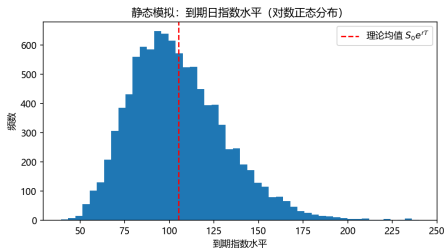
- r : 无风险利率; σ : 波动率; z : 标准正态随机变量。
- 抽一个 z , 就得一个可能的 S_T ; 抽一万个 z , 就得到 S_T 的整个分布。
- S_T 呈**对数正态分布**: 恒为正、右偏 (暴涨的可能性拖出长尾)。

零基础提醒

`exp` 就是指数函数 e^x (`np.exp`)。这个公式不用会推, 会照着写代码就行。

```
1 S0, r, sigma, T = 100.0, 0.05, 0.25, 1.0    # 参数
2 I = 10000                                     # 模拟次数
3 np.random.seed(1000)
4 ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T
5             + sigma * np.sqrt(T) * npr.standard_normal(I))
6 ST.mean()    # 104.86
```

- 一行表达式模拟 10000 个到期价格——**向量化**的威力（第 8 章已见）。
- 理论均值 $S_0 e^{rT} = 105.13$ ，模拟得到 104.86——抽样误差之内，吻合。



- 大部分路径落在 100 元附近，但右尾拖得很长：10000 条里最高冲到约 240 元，最低约 39 元。
- 股价不会为负（指数函数恒正）——对数正态分布的天然优点。
- 只关心到期日？静态模拟一步搞定。但期权若**中途**有特殊条款（如提前行权、触碰障碍），就必须知道**整条路径**——下一节的动态模拟。



1. 为什么需要模拟
2. 随机数
3. 模拟随机变量：一步到位
- 4. 模拟随机过程：走出路径**
5. 欧式期权的蒙特卡洛估值
6. 小结与练习



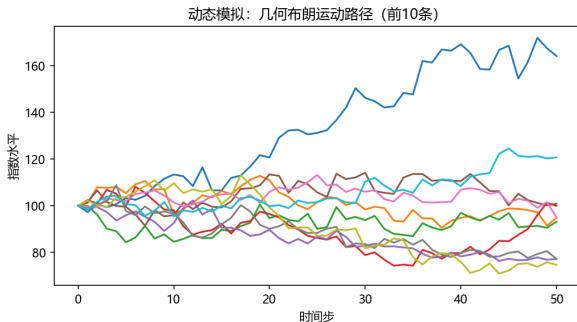
- **随机过程**：一串前后关联的随机变量。股价逐日演化就是典型。
- 把一年切成 $M = 50$ 小步，每步 $\Delta t = T/M$ ，逐步推进：

$$S_t = S_{t-\Delta t} \exp\left(\left(r - \frac{\sigma^2}{2}\right) \Delta t + \sigma \sqrt{\Delta t} z_t\right)$$

- 每一步抽**新的**标准正态随机数 z_t ，乘到当前价格上。
- 这种逐步推进的写法叫**欧拉离散化**——听起来高深，其实就是一个 for 循环。
- 马尔可夫性质：明天的价格只取决于今天的价格，与“怎么走到今天”无关（无记忆）。

```
1 M = 50                      # 时间步数
2 dt = T / M                  # 每步长度
3 I = 10000                   # 路径条数
4 np.random.seed(1000)
5 S = np.zeros((M + 1, I))    # (51, 10000) 的大数组
6 S[0] = S0                   # 所有路径从100出发
7 for t in range(1, M + 1):
8     S[t] = S[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt
9         + sigma * np.sqrt(dt) * npr.standard_normal(I))
```

- 循环只有 50 次（时间步），每步内部 10000 条路径一次算完。
- `S[-1].mean()` 得到 104.60——与静态模拟、理论值一致。



- 每条路径都是“一种可能的未来”：有的一路向上，有的中途跳水。
- 动态模拟的额外收获：可以看清路径中途发生了什么，这是美式期权、障碍期权定价的基础。

- 之前看到：一百万个标准正态随机数的均值也不是精确的 0。
- 两招小手段可以强制达标：

```
1 sn = npr.standard_normal(5000)
2 # 技巧1: 对偶变量——把相反数也加进来, 均值必为0
3 sn = np.concatenate((sn, -sn)) # mean() = 0.0
4 # 技巧2: 矩匹配——直接标准化, 均值0、标准差1
5 sn = (sn - sn.mean()) / sn.std() # std() = 1.0
```

- 随机数的“前两个矩”（均值、标准差）摆正后，模拟结果的波动更小、更稳定——故称**方差缩减**。

零基础提醒

课堂上先建立直觉即可；实务中这两个技巧常与蒙特卡洛定价配套使用。

1. 为什么需要模拟
2. 随机数
3. 模拟随机变量：一步到位
4. 模拟随机过程：走出路径
- 5. 欧式期权的蒙特卡洛估值**
6. 小结与练习



- 风险中性定价公式的蒙特卡洛版本：

$$C_0 \approx e^{-rT} \cdot \frac{1}{I} \sum_{i=1}^I h(S_T^{(i)}), \quad h(S_T) = \max(S_T - K, 0)$$

- 翻译成人话，一共三步：

蒙特卡洛定价三步曲

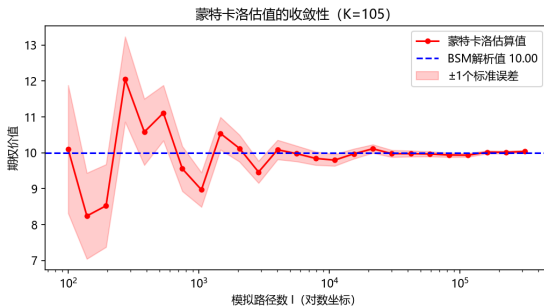
- ① **模拟**：生成 I 个到期价格 $S_T^{(1)}, \dots, S_T^{(I)}$
- ② **算收益**：每条路径到期收益 $\max(S_T - K, 0)$ （没赚就是 0）
- ③ **折现取平均**： $e^{-rT} \times$ 平均收益

```
1 K = 105.0                # 行权价
2 I = 50000                # 模拟路径数
3 np.random.seed(100)
4 sn = npr.standard_normal(I)
5 ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T
6             + sigma * np.sqrt(T) * sn)    # 第1步：模拟
7 hT = np.maximum(ST - K, 0)                # 第2步：收益
8 C0 = np.exp(-r * T) * np.mean(hT)        # 第3步：折现平均
9 print(C0)    # 9.9626
```

- BSM 解析公式给出 10.00，模拟得到 9.96，误差仅 -0.4%——五万条路径已经相当准。
- $\text{np.maximum}(ST - K, 0)$ ：到期价超过行权价才赚钱，否则收益为 0。

```
1 def gbm_mcs_dyna(K, option="call"):
2     S = np.zeros((M + 1, I)); S[0] = S0
3     for t in range(1, M + 1):          # 逐步走路程
4         S[t] = S[t-1] * np.exp((r - 0.5*sigma**2)*dt
5             + sigma*np.sqrt(dt)*npr.standard_normal(I))
6     if option == "call":
7         hT = np.maximum(S[-1] - K, 0)    # 看涨收益
8     else:
9         hT = np.maximum(K - S[-1], 0)    # 看跌收益
10    return np.exp(-r * T) * np.mean(hT)
```

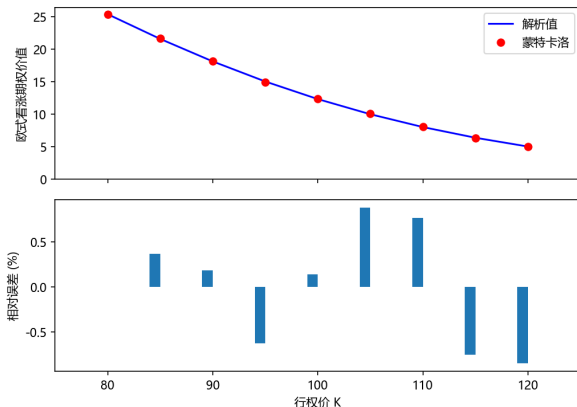
- 实测 ($I = 50000$, $M = 50$): 看涨 10.00 (解析 10.00), 看跌 9.92 (解析 9.88)。
- 欧式期权用动态模拟“绕了远路”, 但这是以后给美式期权定价的脚手架。



实测收敛过程 (标准误差 = 标准差 / $\sqrt{I}$)

路径数 I	500	5,000	50,000	500,000
估算值	10.35	9.94	9.97	9.97
标准误差	0.85	0.24	0.08	0.02

- 误差按 $1/\sqrt{I}$ 缩小：精度提高 10 倍，样本要多 100 倍——这就是蒙特卡洛“用算力换精度”的代价。



- 行权价从 80 到 120，蒙特卡洛估算与解析值的最大相对误差不到 1% (实测 0.88%)。
- 深度实值 ($K=80$) 与深度虚值 ($K=120$) 两端误差略大——收益分布越“偏”，需要的路径越多。

1. 为什么需要模拟
2. 随机数
3. 模拟随机变量：一步到位
4. 模拟随机过程：走出路径
5. 欧式期权的蒙特卡洛估值
- 6. 小结与练习**



- **随机数**: `npr.seed` 保证可重现; `rand/standard_normal` 是两大基础, 线性变换可改变区间。
- **静态模拟**: 一步模拟到期价格 $S_T = S_0 \exp((r - \sigma^2/2)T + \sigma\sqrt{T}z)$, 对数正态分布。
- **动态模拟**: 欧拉格式逐步推进, 循环走时间、向量化走路径; 是路径依赖产品定价的基石。
- **方差缩减**: 对偶变量让均值精确为 0, 矩匹配让标准差精确为 1。
- **蒙特卡洛估值**: 模拟 $\rightarrow$ 算收益 $\rightarrow$ 折现取平均; 误差按 $1/\sqrt{I}$ 收敛, 与 BSM 解析值误差 $< 1\%$ 。

课后任务

- ① 用静态模拟估计**看跌期权**价值 (收益为 $\max(K - S_T, 0)$, $K = 105$), 并与解析值 9.88 对比。
- ② 把波动率从 0.25 提高到 0.40, 重新定价看涨期权, 观察价格变化方向并解释。
- ③ 选做: 分别用 $I = 10^3, 10^4, 10^5, 10^6$ 条路径定价, 记录标准误差, 验证 $1/\sqrt{I}$ 规律。

下章预告

本章的动态模拟写了一个时间循环——模拟一百万条路径时它够快吗？下一章讲**性能**：为什么向量化比循环快，以及如何科学地计时。

本章内容改编自教材第 12 章^[1]。

- [1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京: 人民邮电出版社, 2018.



Thanks for your attention!

Q & A

